System Analysis and design

Part-B

Segment-6 – Testing

Topics: ① Testing defn, objectives
② Test information flow / Testing process
③ Test case design
④ white box testing → Basic path testing
⑤ Flow graph notation
⑥ Node, edge, region defn
⑦ Cyclomatic complexity
⑧ Graph matrices, Connection matrix
⑨ Black box testing
⑩ Alpha, Beta testing
⑪ Alpha Vs Beta testing
⑫ Recovery, Stress, Security, Validation testing

Testing: Testing is basically a process to detect errors in the software product. Testing is a way to know about quality and reliability.

Objective of testing:

① We can define testing as a process of executing a program with the aim of finding errors.
② To perform testing test cases are designed.
③ A test case is a particular made up artificial situation upon which a program is exposed so as to find errors.
④ A good test is one thod finds undiscovered errors.

Testing information flow: Testing is a complete process. For testing, we need two types of inputs.

⇒ 1st is software configuration which includes software requirement specification, design specifications and source code of program.

⇒ 2nd is test configuration. It is basically test plan and procedu

And software configuration is required so that the testers know what is to be expected and tested.

Test configuration is testing plan that is, the way how the testing will be conducted on the system.

~~⇒ Test~~

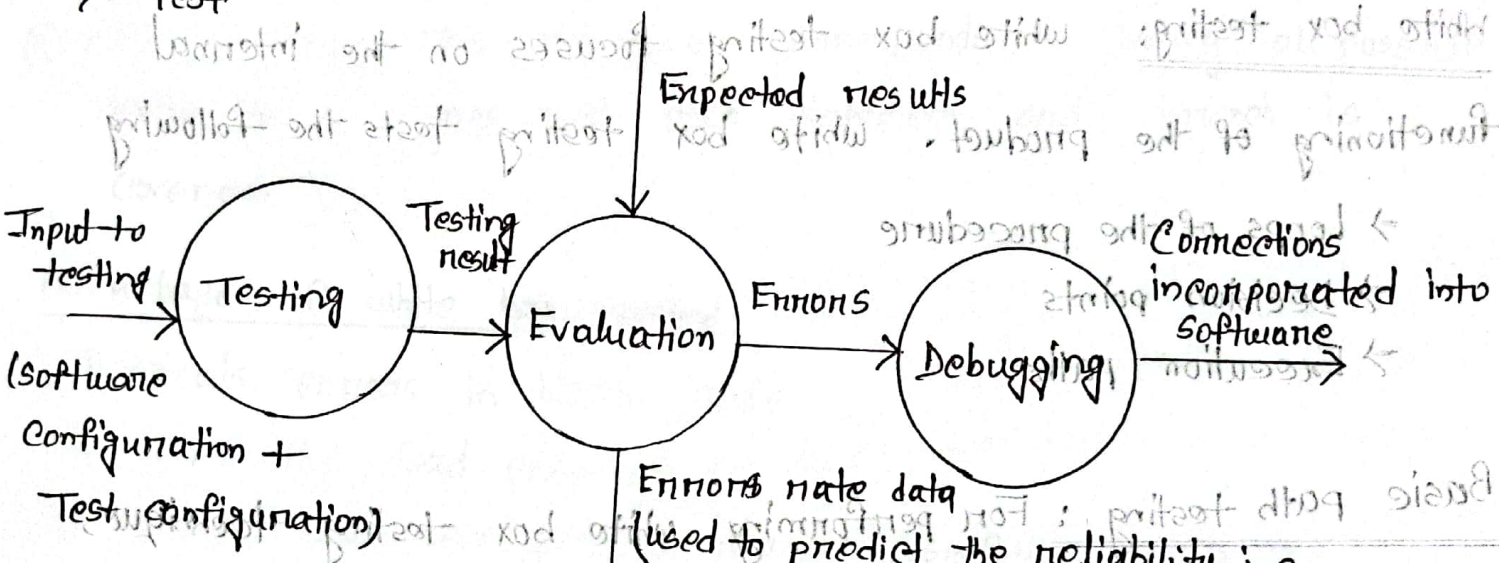


Fig: Testing process

Test case design: Test cases are integral part of testing. We need to know more about test cases and how these test cases are designed. A software product can be tested in two ways.

In 1st approach, the overall functioning of the product is tested. Inputs are given and outputs are checked.

⇒ Black box testing

⇒ white box testing:

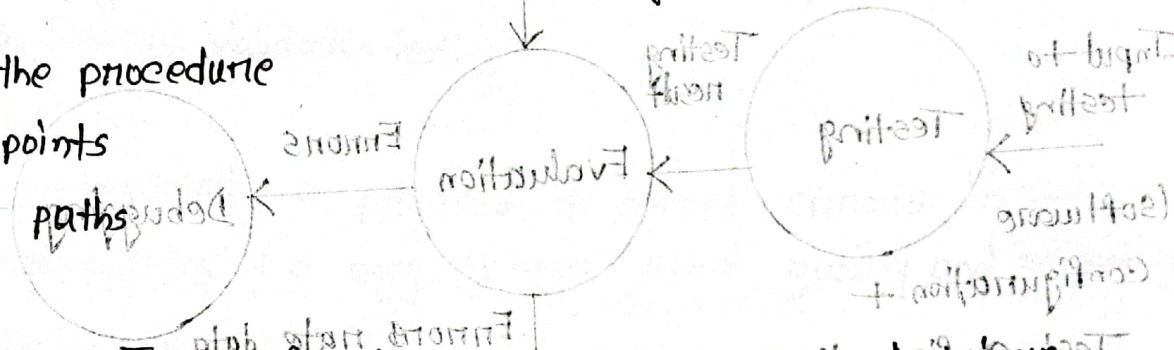
white box testing is more intensive than black box testing.

White box testing: white box testing focuses on the internal functioning of the product. white box testing tests the following

⇒ Loops of the procedure

⇒ Decision points

⇒ Execution paths



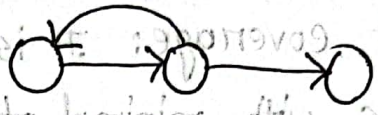
Basic path testing: For performing white box testing technique, basic path testing technique is used. Basic path testing is a white box testing technique. It was proposed by TOM McCabe. These tests guarantee to execute every statement in the program at least one time during testing. Basis set is the set of all the execution path of a Procedure.

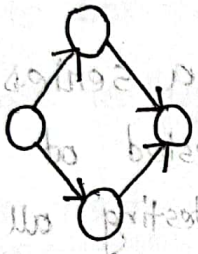
Flow graph notation: simple notation used for the representation of control flow. This notation is known as flow graph.

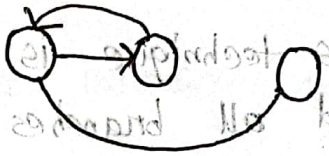
Flow graph depicts control flow and uses the following constructs.

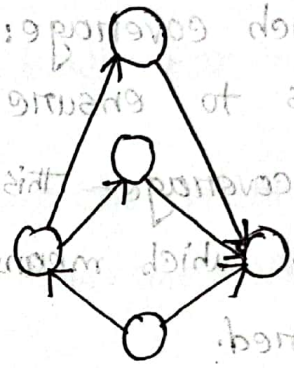
These individual constructs combine together to produce the flow graph for a particular procedure.

Sequence – 

until – 

If – 

while – 

case – 

Node: Each flow graph node represents one or more procedural statements.

Predicate node: Each node that contains a condition is called a predicate node.

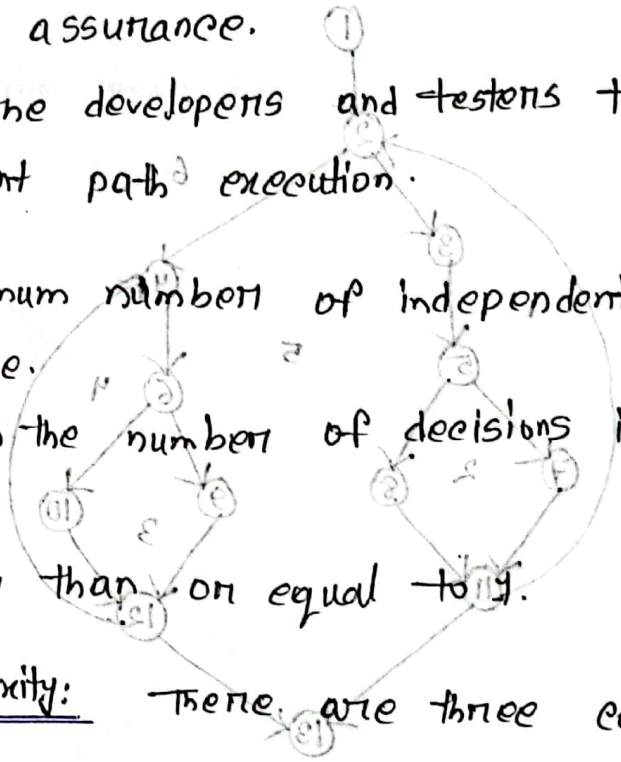
Edge: Edge is the connection between two nodes. The edges between nodes represent flow of control.

An edge must terminate at a node, even if the node does not represent any useful procedural statements.

Region: A region is a flow graph is an area bounded by edges and nodes.

Cyclomatic Complexity: Independent path is an execution flow from the start point to the end point. Cyclomatic complexity provides the number of such execution independent paths.

Importance:
- ⇒ It helps in determining the software quality.
- ⇒ It helps to focus more on the uncovered paths.
- ⇒ It provides assurance.
- ⇒ It helps the developers and testers to determine independent path execution.



Properties:
1. It is the maximum number of independent paths through the program code.
2. It depends only on the number of decisions in the program code.
3. It is always greater than or equal to 1.

Calculating cyclomatic complexity: There are three commonly methods:

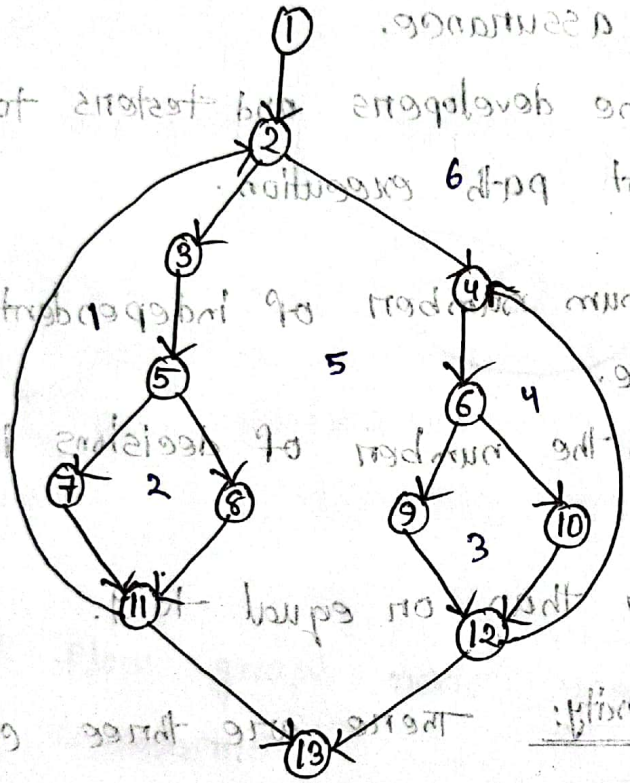
1. Cyclomatic complexity = Total number of regions + 1
2. Cyclomatic complexity, $V(G) = E - N + 2$

   where, E = Number of flow graph edges
   N = Number of flow graph nodes
3. Cyclomatic complexity, $V(G) = P + 1$

   where, P = Total number of predicate nodes contained in the flow graph.

- Predicate nodes are the conditional nodes.
- They give rise two branches in the control flow graph.

Example: Consider the following Graph:



Calculating cyclomatic complexity:

Here, Region = 6, Number of nodes = 13, Number of edges = 17

Number of predicate nodes = 5

Now, cyclomatic complexity, V(G)

$\therefore V(G) = E - N + 2$

$= 17 - 13 + 2$

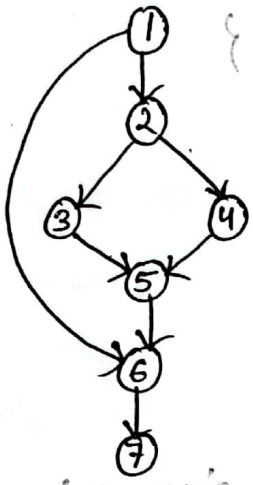
$= 6$

$\therefore V(G) = P + 1 = 5 + 1 = 6$

Problem-1 Calculate cyclomatic complexity for the given code-

```
1. IF A = 354
2.     THEN IF B>C
3.             THEN A = B
4.             ELSE A = C
5.     END IF
6. END IF
7. PRINT A
```

Solution: Control flow graph for given code,



Here, nodes = 7

edges = 8

Predicate nodes = 2 — How?

Number of regions = 2 — How?

Method : 1 – cyclomatic complexity = Total number of regions + 1

= 2+1

= 3

Method-2: V(G) = E-N+2

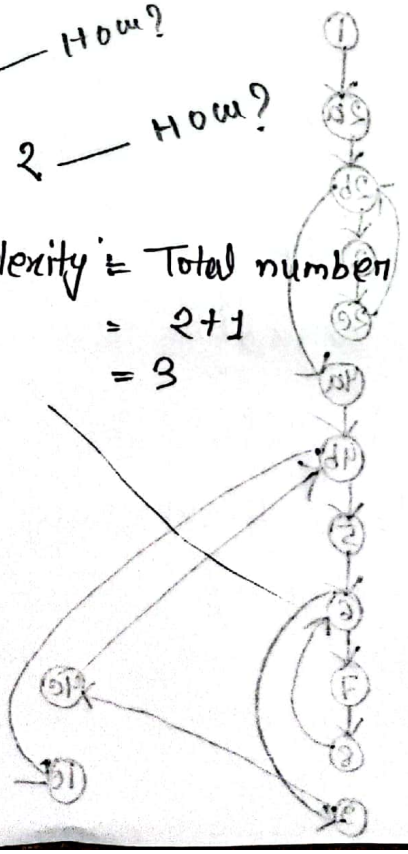
= 8-7+2

= 3.

Method-3: V(G) = P+1

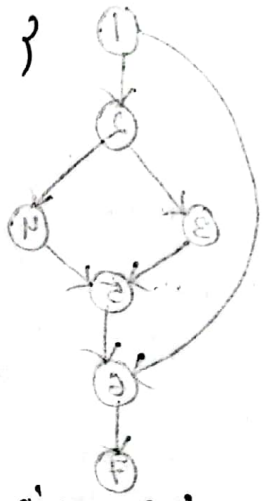
= 2+1

= 3.

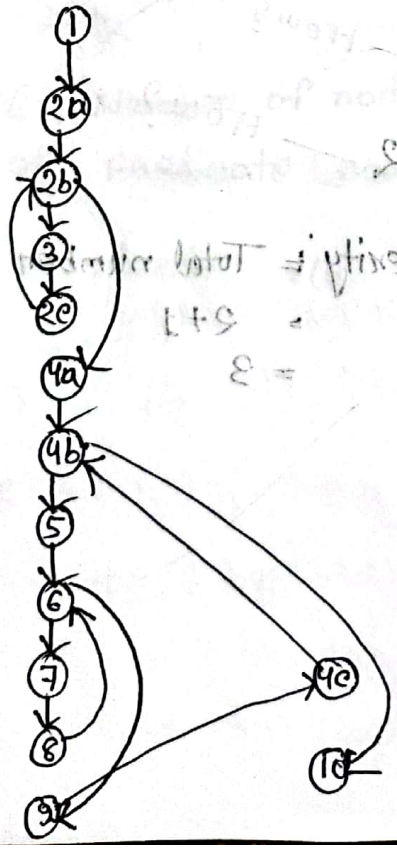


Problem-2 calculate cyclomatic complexity for the given code:-

```
1.  { int i, j, k;
2.    for (i=0 ; i<=N; i++)
3.    P[i] = i;
4.    for (i=2 ; i<= N ; i++)
5.    {
6.        k = P[i]; j = i;
7.        while (a[P[j-1]] > a[k] )
8.            P[j] = P[j-1];
9.            j --;
10.   }
11.       P[j] = k;
12. }
```



Solution: Control flow graph for given code:



Region – 3

Node – 14

edge – 16

Predicate nodes – 3

Method-1:

cyclomatic complexity = Total number of region + 1

= 3+1

= 4

Method-2: cyclomatic complexity = E - N + 2

= 16 - 14 + 2

= 4

Method-3 cyclomatic complexity = P + 1

= 3 + 1

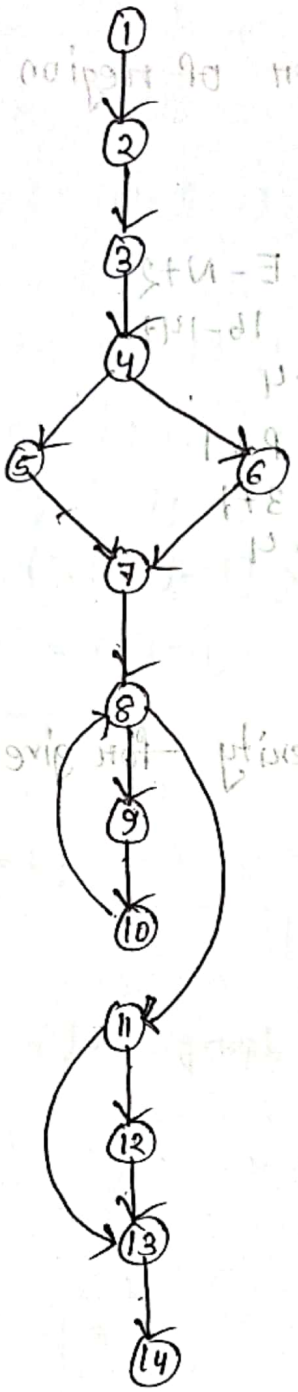
= 4



Problem-3 calculate cyclomatic complexity for given code:

```
1.  begin int x, y, power;
2.        float z;
3.        input (x,y);
4.        if (y<0)
5.        power = -y;
6.        else power = y;
7.        z = 1;
8.        while (power != 0)
9.        { z = z * x;
10.           power = power - 1.
11.       } if (y<0)
12.       z = 1/z;
13.       output (z);
14.       end
```

Solution:



Regions – 3

Nodes = 14

Edges – 16

Predicate node = 3

① cyclomatic complexity = Total number of regions + 1
= 3 + 1
= 4

② V(G) = E – N + 2 = 16 – 14 + 2 = 4

③ V(G) = P + 1 = 3 + 1 = 4.

Deriving test cases:

① From the design or source code, derive a flow graph.

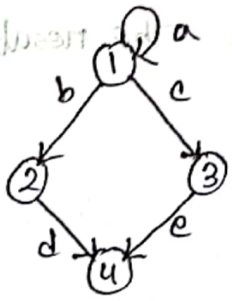
② Determine cyclomatic complexity V(G)

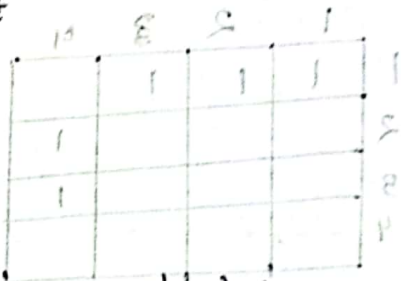
③ Prepare the test cases that will force execution of each path in the basis set.

Graph Matrices: A Graph matrix is a two dimensional matrix. It has rows and columns each equal to number of nodes in flow graph.

A graph matrix is a data structure that can assist in developing a software tool for the automation of basis path testing. A graph matrix is a square matrix whose size is equal to the number of nodes on the flow graph.

Example:





Lets convert this control flow graph into a graph matrix. Since the graph has 4 nodes, so the graph matrix would have a dimension of 4x4. Matrix entries will be filled as follows:

* (1,1) will be filled with 'a' as an edge exists from node 1 to 1.
* (1,2) will be filled with 'b' as an edge exists from node 1 to node 2

	1	2	3	4
1	a	b	c	
2				d
3				e
4				

(*) (1,3) will be filled with e as edge e exists from node 1 to node 3.

(*) (2,4) will be filled with 'd' as an edge exists from node 2 to 4.

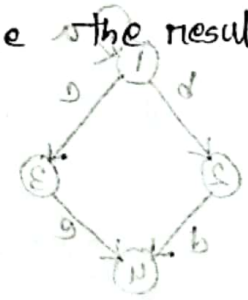
(*) (3,4) will be filled with 'c' as an edge from node 3 to node 4.

Connection matrix: A connection matrix is a matrix defined with edges weight. When a connection exists between two nodes of control flow graph then the weight is 1, otherwise it is 0.

However, 0 is not usually entered in the matrix cells to reduce the complexity.

For example, if we represent the above control flow graph as a connection matrix, the result would be,

	1	2	3	4
1	1	1	1	
2				1
3				1
4				



A connection matrix is used to find the cyclomatic complexity of the control graph.

Steps to compute the cyclomatic complexity:

① Count the number of 1s in each row and write it in the end of the row.

② Subtract 1 from this count for each row

③ Add the count of each row calculated prieviously.

④ Add 1 to this total count.

⑤ The final sum in Step 4 is the cyclomatic complexity of the control flow graph.

Lets apply this steps to the graph above to compute the cyclomatic complexity.

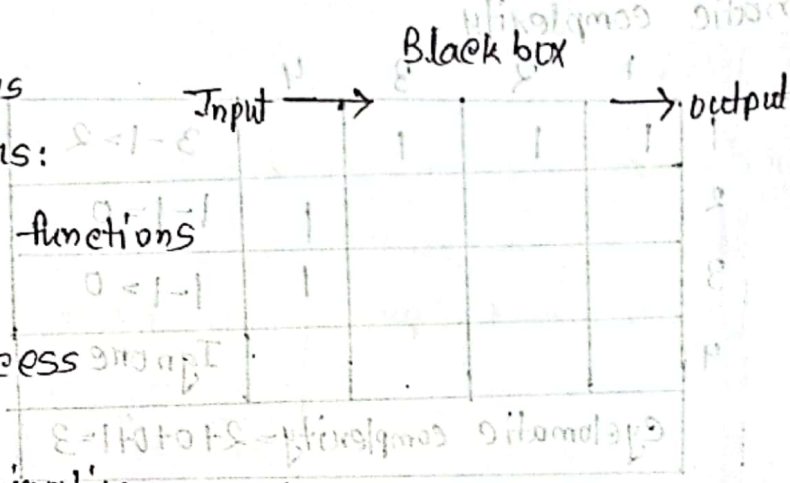
	1	2	3	4	
1	1	1	1		3-1=2
2				1	1-1=0
3				1	1-1=0
4					Ignore
Cyclomatic complexity = 2+0+0+1=3					

we can verify this value for cyclomatic complexity using other methods also.

☐ Cyclomatic complexity indicates several information about program code:

- 1-10
  - ⇒ Structured and well written code
  - ⇒ High testability
  - ⇒ Less cost and effort
- 10-20
  - ⇒ Complex code
  - ⇒ Medium Testability
  - ⇒ Medium cost and effort
- 20-40
  - ⇒ Very complex code
  - ⇒ Low Testability
  - ⇒ High cost and effort
- >40
  - ⇒ Highly complex code
  - ⇒ Not at all Testable
  - ⇒ Very high cost and effort

Black box testing: Black box testing is a technique of software testing which examines the functionality of software without peering into its internal structure of coding. Black box testing test the overall functional requirements of product.



Black box testing uncovers following types of errors:

⇒ Incorrect or missing functions
⇒ Interface errors
⇒ External database access
⇒ Performance errors
⇒ Initialization and termination errors.

There are two technique during black box testing:

① Equivalence partioning
② Boundary Value analysis

Equivalence partitioning: Equivalence partitioning or equivalence class partitioning is type of black box testing technique which can be applied to all levels of software testing like unit, integration, system etc

⇒ Input data units or input domains are divided into classes or groups of data. These classes are known as equivalence classes and the process of making equivalence classes is called equivalence equivalence partitioning.

Example:

⇒ If an input condition specifies a range, one valid and two invalid equivalence classes are defined.

⇒ If an input condition requires a specific value, then one valid and two invalid equivalence classes are defined.

⇒ If an input condition specifies a number of a set, then one valid and one invalid equivalence classes are defined.

⇒ If an input condition is Boolean, then one valid and one invalid equivalence classes are defined.

Example: Let's consider the behaviour of order pizza text box below

- Pizza values 1 to 10 is considered valid. A success message is shown.
- while value 11 to 99 are considered invalid for order and an error message will appear "only 10 pizza can be ordered".

Order pizza: [ ] Submit

Invalid	Valid	Invalid	Invalid
0	1 10	11 99	100
Partition 1	Partition 2	Partition 3	Partition

Invalid	Valid	Invalid	Invalid
0	1 10	11 99	10 0
Partition 1	Partition 2	Partition 3	Partition 4
-1	5	88	121

If any one value from the set passes the test then the whole set of Partition is considered pass or Valid.

Boundary value analysis: Boundary value test cases are called extreme cases. Boundary value test case is a set of input data that lies on the edge or boundary of a class of input data or that generates output that lies at the boundary of a class of output data.

Example: The boundary value analysis we test boundaries between equivalence partitions.

Invalid	Valid	Invalid	Invalid
0	1 10	11 99	100
Partition 1	Partition 2	Partition 3	Partition 4

↓

In boundary value analysis, we will check the boundary values like 0, 1, 10, 11, 99, 100

Black box testing Advantage:

① Easy to automate.

② It results in the optimization of code removing error and helps in removing extra line of code.

Disadvantage:

① Very expensive

② Missing functionalities can't be detected.

③ Very complex and at times not realistic.

Alpha testing: Alpha testing is a type of acceptance testing. It performed to identify all possible issues before releasing the product to everyday users on public.

Unit testing → Integration testing → system testing → Alpha testing → Beta testing

user acceptance testing

Beta testing: Beta testing of a product is performed by real users of the software application in a 'real environment' & can be considered as a form of external user acceptance testing.

Alpha testing vs Beta testing:

Alpha testing	Beta testing
① Performed by testers.	① Performed by clients or end users.
② Performed at developer's site.	② Performed at client location
③ Requires lab environment or testing environment.	③ Doesn't require any lab environment or testing environment.
④ Reliability and security testing are not performed in depth alpha testing.	④ Reliability, security, are checked during Beta testing.
⑤ Alpha testing involves both white box and black box testing.	⑤ Beta testing typically uses black box testing.

Recovery testing: Recovery testing is a system that forces the software to fail in a variety of ways and verifies that recovery is properly performed. The recovery is automated re-initialization mechanisms, data recovery, and restart are each evaluated for correctness.

Stress testing: stress testing is a form of deliberately intense or through testing used to determine the stability of a given system or entity.

Reasons:

⇒ To determine breaking points or safe usage limits.

⇒ To confirm mathematical model is accurate enough in Predicting breaking points or safe usage limits.

⇒ To confirm intend specifications are being met.

⇒ To determine modes of failure.

⇒ To test stable operation of a part or system outside standard usage.

Security testing: Security testing attempts to verify that protection mechanism built into a system will protect it from unauthorized penetration. During security system the teston plays the role of the individual who desires to penetrate the system. The testori may attack the system with custom software designed to break down any defenses that have been constructed. Good security ~~syst~~ testing will ultimately penetrate a system.

Validation testing: Validation testing is the process of assessing a new software product to ensure that its performance matches Consumer needs. Validation succeeds when software functions in a manner that can be expected by the customer. After each validation test case has been conducted, one of two possible conditions exists:

The function or performance characteristics confirm to specification and are accepted or A deviation from specification is uncovered and a deficiency list is created.